

Development Documentation

Development Documentation — OptiCrop

1. Environment Setup

- Python 3.10+
- Virtual environment configured for dependency isolation
- Required libraries: Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, Pickle, Jupyter Notebook, HTML/CSS/Bootstrap, Git

2. Development Steps

Step 1 — Data Preprocessing & Cleaning

- Load dataset containing N, P, K, temperature, humidity, pH, rainfall, and crop label
- Handle missing values
- Encode categorical target (crop label) if needed
- Feature scaling (e.g., StandardScaler) where required by the algorithm

Step 2 — Model Selection & Training

Implemented and compared the following algorithms: - K-Nearest Neighbors (KNN) - Logistic Regression - Decision Tree - Random Forest - K-Means Clustering (for exploratory grouping/analysis)

Each model was trained on the same train/test split and evaluated using accuracy and other relevant classification metrics. The best-performing model was selected for deployment.

Step 3 — Model Persistence

- Best model serialized using Pickle (`model.pkl`) for reuse without retraining on every prediction request.

Step 4 — Backend Development (Flask)

Modules built: - Data preprocessing/input validation - Model loading and prediction function - Flask routes:
- `GET /` → renders input form - `POST /predict` → validates input, runs prediction, renders result

Step 5 — Frontend Development

- Input form built with HTML/CSS/Bootstrap for N, P, K, temperature, humidity, pH, rainfall
- Result page dynamically displays the predicted crop
- Responsive design for desktop and mobile browsers

3. Folder Structure

```
OptiCrop/  
├── Dataset/  
├── Flask/  
│   ├── app.py  
│   ├── model.pkl  
│   └── templates/
```

```
| | | └─ index.html
| | | └─ result.html
| | └─ static/
| |   └─ css/
└─ Training/
    └─ model_training.ipynb
└─ Project_Documentation/
    requirements.txt
└─ README.md
└─ .gitignore
```

4. Version Control

- Git used for source control
- GitHub repository maintained with regular commits documenting progress (e.g., dataset cleanup, documentation updates)

5. Known Development Notes

- Model retraining requires re-running the training notebook and replacing `model.pkl`
- Input validation on the Flask side prevents invalid/out-of-range values from reaching the model